OpenGL Assignment 2:
Shading and Texture Mapping

September 28, 2026

Computer Graphics

Information

This assignment consists of two parts: a theoretical part (3 points) and a practical part (8 points). The theoretical part should be submitted as a PDF on Brightspace.

The practical part will be submitted through Themis, available on `https://themis.housing.rug.nl`. Before submitting your code, **make sure to enroll in a group with your partner first!** You absolutely need to do so before submitting any code: old submissions will not be transferred to the group and this will clutter the view when grading. Only in exceptional cases can you change groups, but only after informing the teaching assistants beforehand.

Working with Themis

This assignment consists of multiple sub-assignments in which you will be repeatedly adding new functionality to a given framework. After completing a sub-assignment, compress your project into a `zip` or `tar.gz` file and submit it to the corresponding sub-assignment on Themis. Include all files and folders used, **excluding** `CMakeLists.txt.user` files and the build folder. For Themis' judgment to work properly, it is also important that you do not modify the signature of existing `public` methods.

It may take a minute for Themis to judge your submission, especially if a sub-assignment has many test cases. After a submission, you will have to wait 5 minutes before making another submission for that sub-assignment. You have unlimited attempts at each sub-assignment. The sub-assignments are intended to be completed in order, and submitting a later solution to an earlier sub-assignment may not necessarily work.

Themis checks whether your code produced the correct output. After the deadline, the teaching assistants will use the latest attempt for each sub-assignment to determine your assignment grade (which may be different from the one Themis shows).

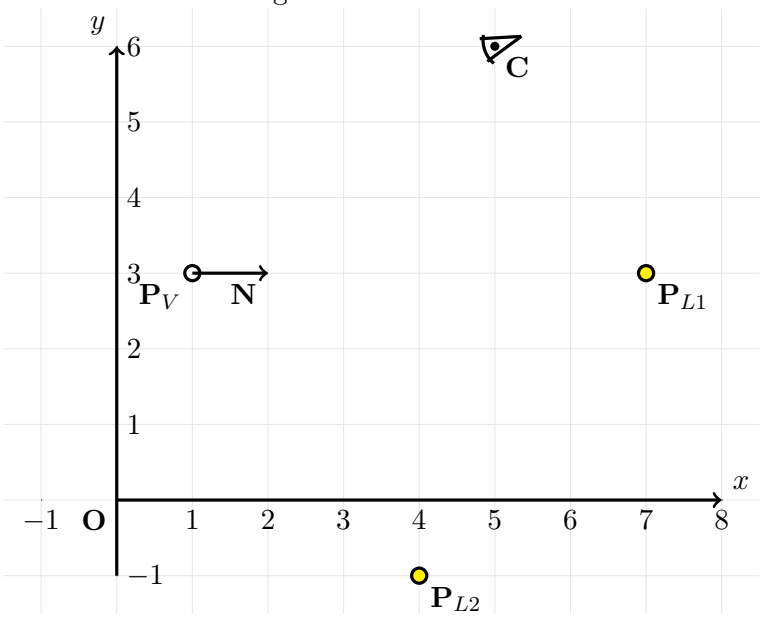
Some sections contain suggestions for the competition. Please do not include these in your submission, as it may affect Themis' judgment.

1 Theoretical question (3 points)

This theoretical part should be submitted as a single PDF file on Brightspace. For your submissions, you should use LaTeX. For drawings/diagrams, consider using Draw IO, InkScape or TikZ. However, you can also include hand-drawn diagrams (provided that they are legible).

Remember to use exact values in your answers, i.e. $\frac{1}{3}\sqrt{2}$, not 0.4714...

Consider the following scene:



We have a vertex with the following properties:

- Coordinates: $\mathbf{P}_V = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$
- Normal: $\mathbf{N} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
- Color (RGB): $\mathbf{V}_{col} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$
- Ambient coefficient: $k_a = 0.2$
- Diffuse coefficient: $k_d = 0.8$
- Specular coefficient: $k_s = 0.2$
- Specular exponent: $p = 1$

We have two point light sources with the following properties:
Light source 1:

- Coordinates: $\mathbf{P}_{L1} = \begin{pmatrix} 7 \\ 3 \end{pmatrix}$
- Color (RGB): $\mathbf{L}_{1,col} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$

- Intensity: $\mathbf{I}_{1,a} = \mathbf{I}_{1,d} = \mathbf{I}_{1,s} = 0.5$

Light source 2:

- Coordinates: $\mathbf{P}_{L2} = \begin{pmatrix} 4 \\ -1 \end{pmatrix}$
- Color (RGB): $\mathbf{L}_{2,col} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$
- Intensity: $\mathbf{I}_{2,a} = \mathbf{I}_{2,d} = \mathbf{I}_{2,s} = 0.5$

The camera is positioned at coordinates $C = \begin{pmatrix} 5 \\ 6 \end{pmatrix}$

1. **(1.25 points)** Compute the resulting color of the vertex, as seen by the camera using the Phong reflection model. (Assume the ambient intensity is obtained by averaging the ambient intensities of the light sources)
2. **(0.45 points)** Which component of the Phong reflection model would be used to model a mirror?
3. **(0.45 points)** Which component of the Phong reflection model would be used to model a wooden material?
4. **(0.85 points)** What is the difference between the Phong reflection model and the Blinn-Phong reflection model? When would one choose to use the Blinn-Phong model over the Phong model?

2 OpenGL implemenation

The assignment uses a different framework from OpenGL 1, so make sure to download the correct OpenGL 2 framework from Brightspace/GitLab. This framework will render a single cat model with random colors at its vertices.

In this assignment you will implement a set of shading techniques in such a way that your program can easily swap between them.

3 Shaders

In the previous assignment, you already implemented a minimal shader program to start with. Now, we will create a set of new shader programs that can render a mesh differently. In the UI, you will be able to switch between shader programs in real-time.

Normal Shader (1 point)

The first new shading method your program will support is a simple shader that will visualize the normals of your mesh. This method computes a color at each vertex using the normal of that vertex. These colors are then interpolated across the surface of the triangle. The fragment shader will receive the interpolated color as a user-defined input (e.g. `in vec3 vertNormal`).

1. Our vertices will no longer store an RGB color, but will store an xyz-normal instead. Change the naming of the vertex color related variables in all the C++ and GLSL code to refer to normals instead. Test to see if your application still works (the changes should only be semantic).

    - Remember that you can pass variables from the vertex shader to the fragment shader by using the **out** qualifier in the vertex shader and **in** qualifier in the fragment shader, the variables *must* have the same name.

2. Our vertices now have a 'normal' attribute, but it still stores a color. In order to render the normals of the mesh, we will need to obtain the normals from our model file and store them in the vertices. Obtain a `QVector` containing the normals corresponding to the coordinates by calling `getNormals()` on the `model` instance. This is analogous to how the vertex coordinates are obtained. Replace the random color data transferred to the color buffer with the normal vectors and rename all associated variables in `mainview.h` and `mainview.cpp` to an appropriate name reflecting the data the variables contain. [†]

3. Create a `QMatrix3x3` instance used for the normal matrix. We will use this matrix for transforming the normals, which needs to be done in order to preserve their relative angles when the model gets transformed. The transformation of the normals themselves will take place in the vertex shader.
   Please refer to the lecture slides as to how a normal transformation matrix is constructed and initialize the model transformation matrix. *Hint*: Use the method `QMatrix4x4::normalMatrix()` to obtain the normal matrix from the model/mesh transformation matrix. Make sure the normal matrix is updated properly whenever the transformation matrix updates.

[†] Renaming the variables is not required for the code to work, but it makes the code more readable.

4. Make the normal transformation matrix available in the vertex shader following the usual steps. That is: introduce a uniform variable for this matrix in the vertex shader, and in the method `paintGL`, locate where the uniform values are set, and invoke the appropriate form of the `QOpenGLShaderProgram::setUniformValue` method with a name matching the variable name in the vertex shader.

5. In the vertex shader, transform the normal using the normal transformation matrix. *Normalize* the result and pass it to the fragment shader.

6. The fragment shader should now receive an interpolated normal. *Normalize* it, map it to a color (see the lecture slides on how to do this), and write the result to the output color.

7. The result should look like Figure 3.1.

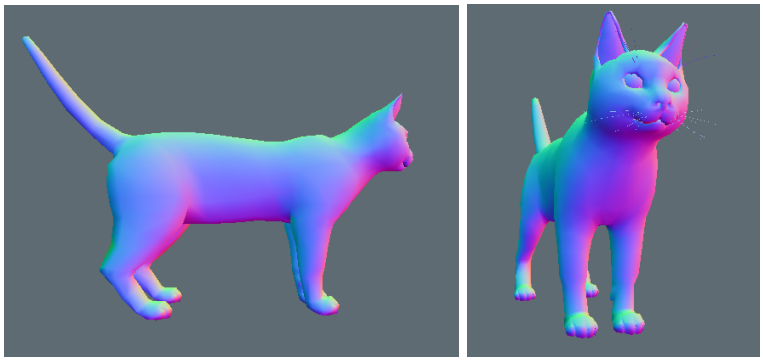


Figure 3.1: *The visualized normals of a cat.*

Themis Submission

Once you have completed this part, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

Selecting Shaders (1 point)

Before implementing any additional shaders, we should make sure we can switch between them.

1. Go to the folder containing the shaders, and rename `vertshader.glsl` and `fragshader.glsl` to the more appropriate `vertshader_normal.glsl` and `fragshader_normal.glsl` respectively. Then, make two duplicates of them and rename these to the following:

   - `vertshader_gouraud.glsl` and `fragshader_gouraud.glsl`.
   - `vertshader_phong.glsl` and `fragshader_phong.glsl`.

2. Change the output color of the Phong and Gouraud fragment shaders so that you can easily see the difference between the different shader programs. Specifically, make the Phong shader output a uniform green color (0,1,0) and the Gouraud shader a uniform blue color (0,0,1).

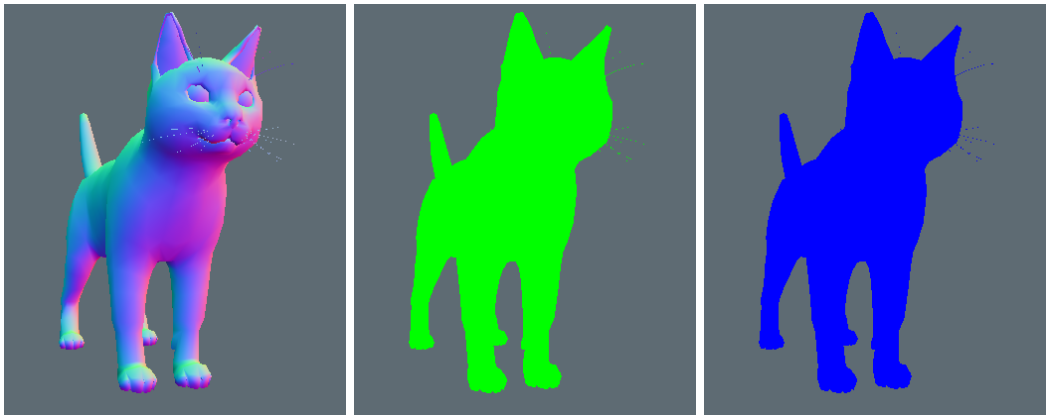
3. Import the new set of shaders into Qt. You can do this by right-clicking the `shaders` folder and pressing "add existing files". If you duplicated the files via QtCreator itself, this step might not be necessary.

4. Create two new shading programs so that you can use the Gouraud/Phong shaders. *Hint*: It is possible to replace the `shaderProgram` variable with an array with 3 elements. Use and/or modify the values of the provided `ShadingMode` enum, so it can be used to select a program from the array.

5. Change the `paintGL` method to modify the uniform variables of the currently selected shader program using the `QOpenGLShaderProgram::setUniformValue()` method. If the current program does not have a uniform with the specified name, the call will have no effect, and it won't produce any errors.[†]

6. Implement the `setShadingMode` function so that your program can switch between the shaders.

7. Switch between the shader programs to check if your addition was successful. The result should look like Figure 3.2.



(a) *Normal Shader* ***(b)*** *Phong Shader* ***(c)*** *Gouraud Shader*

Figure 3.2: *The different (temporary) shaders on the cat model.*

Themis Submission

Once you have completed this part, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

[†]Lack of error messages is usable when handling multiple programs with different uniforms, but it might cause issues if the name in the vertex shader and the name given as the first argument differ due to a typo. Make sure to copy the variable name exactly.

Gouraud Shader (1 point)

The second new shading method your program will support is called Gouraud shading. This method calculates the color intensity at each vertex using the normal of that vertex and the direction to a light. These intensities are then interpolated before they are received by the fragment shader.

1. Implement Gouraud shading in `vertshader_gouraud.glsl` and `fragshader_gouraud.glsl`. Keep the following in mind:
   - Use the following properties for the material:
     - Material color: (1,1,1).
     - Ambient coefficient: $k_a = 0.4$
     - Diffuse coefficient: $k_d = 0.4$
     - Specular coefficient: $k_s = 0.4$
     - Specular exponent: $p = 8$
   - Use the following properties for the light:
     - Light position: (100,50,0)
     - Light color: (1,1,1)
   - You will need an extra uniform and/or constants for the light position, the material coefficients and - optionally - you may add a uniform for the color of the light. Add a constant for the material color (this will be replaced by the texture color in the next task). If you want, you can add widgets in your UI to modify these uniforms. Make sure to keep the above values as defaults to ensure that your Themis submission will still work properly!
   - Keep the light position fixed in the scene. When rotating the model, the light should not change its position. In other words, don't apply any transformations to your light position.
   - Calculate the color using the Phong Illumination Model in the vertex shader and send it to the fragment shader.
2. The result should look like Figure 3.3 (depending on the rotation of the model of course).

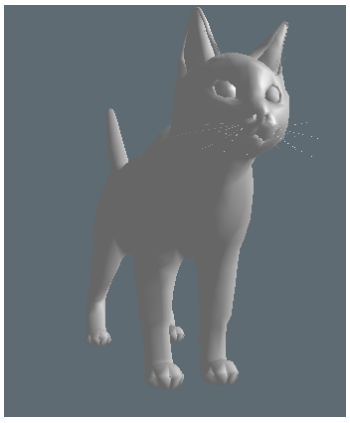


Figure 3.3: *A shiny Gouraud shaded cat.*

Themis Submission

Once you have completed this part, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

Phong Shader (2 points)

Phong shading improves on Gouraud shading by calculating the lighting for each pixel of the triangle fragment, instead of computing the lighting intensity of each vertex and interpolating the result.

1. Implement Phong shading in `vertshader_phong.glsl` and `fragshader_phong.glsl`. You could regard this as taking the Gouraud shading implementation and moving the illumination model from the vertex shader to the fragment shader.

2. In order to access the position of the fragment in world/view coordinate system, pass the position of the vertex transformed only by the `modelViewTransform` as an output attribute in the vertex shader. This will become interpolated input attribute in the fragment shader.

3. Use the same material constants, light position and light color as specified for Gouraud shading. †

4. The result should look like Figure 3.4 (depending on the rotation of the model of course).

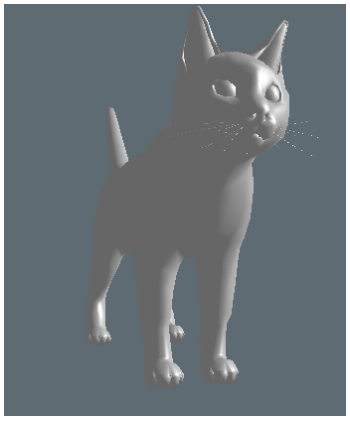


Figure 3.4: *A shiny Phong shaded cat.*

Themis Submission

Once you have completed this part, submit it to Themis and verify that it passes all test cases. If so, you can move on to the next part.

†Intepolation of the normal won't preserve its size. Don't forget to `normalize` the normal again in the fragment shader.

Suggestions for the competition

If you want to add some additional fun features, you may want to do the following:

- Implent a Z-Buffer shader: https://en.wikipedia.org/wiki/Z-buffering
- Implement a Cel shader: https://en.wikipedia.org/wiki/Cel_shading

4 Texture mapping

Using textures is an easy way to create more detail in objects. Instead of using a single color, an image is mapped to an object, such as the fur color on a cat.

Basic method (3 points)

The idea is to map a square *diffuse texture* (also referred to as a *color texture*) to a mesh.

1. To use a `.png` file as a texture, we first need to be able to read this file into memory. Included in the framework is the file `utility.cpp` containing the function `QVector<quint8> MainView::imageToBytes(QImage image)` which will allow you to convert a `QImage` (constructable with the path to an image) to a simple vector of values.

2. Analogous to working with a VAO, we will need to generate a *texture name* in the form of a `GLuint` variable, which we can later use to refer to this texture.
   Declare a `GLuint` variable in the `Mainview` class, then generate a texture name in `mainview.cpp` by calling `glGenTextures`. Add the corresponding `glDeleteTextures` call as well.

3. To modify or use the generated texture, it needs to be bound using `glBindTexture`.

4. How OpenGL will interpret the texture can be set using `glTexParameteri(GL_TEXTURE_2D, <Parameter Name>, <Parameter Value>)`. Here, we will set these parameters:

   - `GL_TEXTURE_WRAP_S` and `GL_TEXTURE_WRAP_T`, to define the wrapping. Set both of these to `GL_CLAMP_TO_EDGE`.
   - `GL_TEXTURE_MIN_FILTER`, to define the minifying filter. Set this to `GL_LINEAR`.
   - `GL_TEXTURE_MAG_FILTER`, to define the magnifying filter. Set this to `GL_LINEAR`.

   Call the function `glTexParameteri` four times to set these four parameters. See the OpenGL RefPages for more info on the `<Parameter Value>`.

5. With all the parameters set we can upload the data using the `glTexImage2D(..)` function. It takes the following parameters:

   (a) Target texture, `GL_TEXTURE_2D` for simple 2D textures.
   (b) Mipmap level, use 0 for now.
   (c) Internal data format, the format OpenGL will store the pixels in on the GPU. Use `GL_RGBA8` for RGBA values, each 8 bits.
   (d) Texture width, width of the texture image.
   (e) Texture height, height of the texture image.
   (f) Border, always use 0 here.
   (g) Format, the format of the pixels your are sending. Use `GL_RGBA` (*not* `GL_RGBA8`).
   (h) Type of each of the red, green, blue, and alpha channels, use `GL_UNSIGNED_BYTE`.
   (i) Pointer to the data to be uploaded, `<yourvector>.data()` where `<yourvector>` is the QVector of bytes you stored the image in using `imageToBytes()`.

6. **Important!** If you decide to use mipmapping (which is one of the options for the minifying filter), call `glGenerateMipmap(GL_TEXTURE_2D)` after uploading the image or your texture may not appear later.

7. To map your texture to the mesh, the `Vertex` class needs to be extended with an additional attribute for the 2D *texture coordinates*.

   - Add a `QVector2D` variable at the end of the `Vertex` class.
   - In `Mainview::loadMesh()`, obtain the texture coordinates of the model using e.g. the `QVector<QVector2D> Model::getTextureCoords()` function.
   - The texture coordinates will be in attribute location 2. Update the vertex attribute specifications by adding the appropriate calls to `glVertexAttribPointer` and `glEnableVertexAttribArray`.

8. We will implement texture mapping in the Phong shader program, not the Gouraud shader program. The latter should still function as before (and ignore the texture coordinates at location 2).
   Modify the Phong vertex shader to receive the texture coordinates at location 2 and to pass these on to the fragment shader. Note that the fragment shader will receive *interpolated* texture coordinates.

9. In the fragment shader, create a uniform of the type **sampler2D**. This will be used to retrieve the color (a **vec4**) from the texture at the provided coordinates. As with all uniforms, create a `GLint` variable for its location in the `Mainview` class, and retrieve and store its location.

10. Use the following line to retrieve a color from the texture based on the texture coordinates:
    `vec3 textureColor = texture(samplerUniform, textureCoords).rgb`,
    where `samplerUniform` is the name of your sampler uniform and `textureCoords` is a `vec2` of your texture coordinates you retrieved from the vertex shader. Modify the shading calculations to include the texture color (i.e. material color) according the the Phong shading model.

11. In `paintGL()`, call `glActiveTexture(GL_TEXTURE0)`[†] and bind the texture using `glBindTexture(GL_TEXTURE_2D, texPtr)` before your call to the drawing function. Replace `texPtr` with your `GLuint` for the texture.

12. The `sampler2D` uniforms are bound to the current active texture - `GL_TEXTURE0` by default. Set the uniform to `0` using

    ```
    program.setUniformValue(<name>, 0);
    ```

    to specify the texture bound to the default `GL_TEXTURE0`. In case more textures are required, `GL_TEXTURE1` requires the corresponding uniform to have value of `1`, `GL_TEXTURE2` – a value of `2` and so on. *Note*: `GL_TEXTURE0` itself is not equal to `0`.

13. You should now see a Phong shaded texture mapped cat similar to the one in Figure 4.1.

[†]For this assignment, this call is optional as we are using the texture that is bound by default. See the competition suggestions on using multiple textures for a case in which these calls are necessary.



Figure 4.1: *A textured cat.*

Themis Submission

Once you have completed this part, submit it to Themis and verify that it passes all test cases. **Note that your latest submission for this part will be used to judge the code quality and documentation!**

Suggestions for the competition

If you want to add some additional fun features, you might want to do the following:

- Use multiple texture maps (*diffuse* and *bump/normal* maps, or more advanced, *displacement* and *ambient occlusion* or *specular* maps). You need multiple calls to `glActiveTexture` and `glBindTexture` with a different texture number. Also make sure you set the right location of your sampler uniforms!
- Import your own models and textures (see today's slides for a couple of links to repositories)! Note that using texture sizes that are powers of 2 are generally preferred for performance reasons. You can use a tool like Blender (`https://www.blender.org/`) to export other models to the `.obj` file format. If you do this, either triangulate your mesh first, or set the export options to *triangulate faces*.
- Use *anisotropic filtering* on your texture. This will prevent blurring of the texture when your viewing vector is almost tangent to the surface of the textured object. This can be enabled by placing the following code snippet in `loadTexture()` before you upload your texture.

  ```
  GLfloat f;
  glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY_EXT,&f);
  glTexParameterf(GL_TEXTURE_2D,GL_TEXTURE_MAX_ANISOTROPY_EXT,f);
  ```

- Implement the function `unitize()` in the `Model` class. As you might have encountered, the scale of a model is determined by the creator of the model. The provided cube has sides of length 2, whereas the sphere has a diameter of 50. It is also possible that the creator of the model did not center it around the origin of the model. By pre-scaling each model, you can make your program more robust and show each model at a similar scale and set the real size with a scaling transformation. Try to determine the size of the bounding box of your model and uniformly scale it to fit inside the unit cube (a cube with sides of length 1) and center it around the origin.